

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – High (Coding Challenge)

COURSE CODE: CSA09

COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO1: *Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)*

QUESTIONS: 10

Total Marks: 100

ANNEXURE A Coding Challenge

Code of Conduct:

I N. Hemanth Reddy (192472215)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Coding Challenge

1. Library Book Management using Classes & Encapsulation

Design a Book class with private fields such as bookId, title, author, and price. Implement constructors, getters, setters, and methods to issue and return books. Create a menu-driven program to manage multiple books using an array of objects. Also, construct suitable test cases to validate book addition, issue, return, and search operations. BTL: BL3 (Apply)\

CODE;

```
class Book {  
  
    private int id;  
  
    private String title;  
  
    private boolean issued;  
  
    Book(int id, String title) {  
  
        this.id = id;  
  
        this.title = title;  
  
        issued = false;  
  
    }  
  
    public int getId() {  
  
        return id;  
  
    }  
  
    public void issueBook() {  
  
        if (!issued) {  
  
            issued = true;  
  
            System.out.println("Book Issued");  
  
        } else {  
  
            System.out.println("Already Issued");  
  
        }  
  
    }  
  
    public void returnBook() {
```

```

        issued = false;

        System.out.println("Book Returned");

    }

    public void display() {

        System.out.println(id + " " + title + " " + (issued ? "Issued" : "Available"));

    }

}

public class Main {

    public static void main(String[] args) {

        Book[] books = new Book[2];

        books[0] = new Book(101, "Java");

        books[1] = new Book(102, "Python");

        books[0].display();

        books[0].issueBook();

        books[0].display();

        books[0].returnBook();

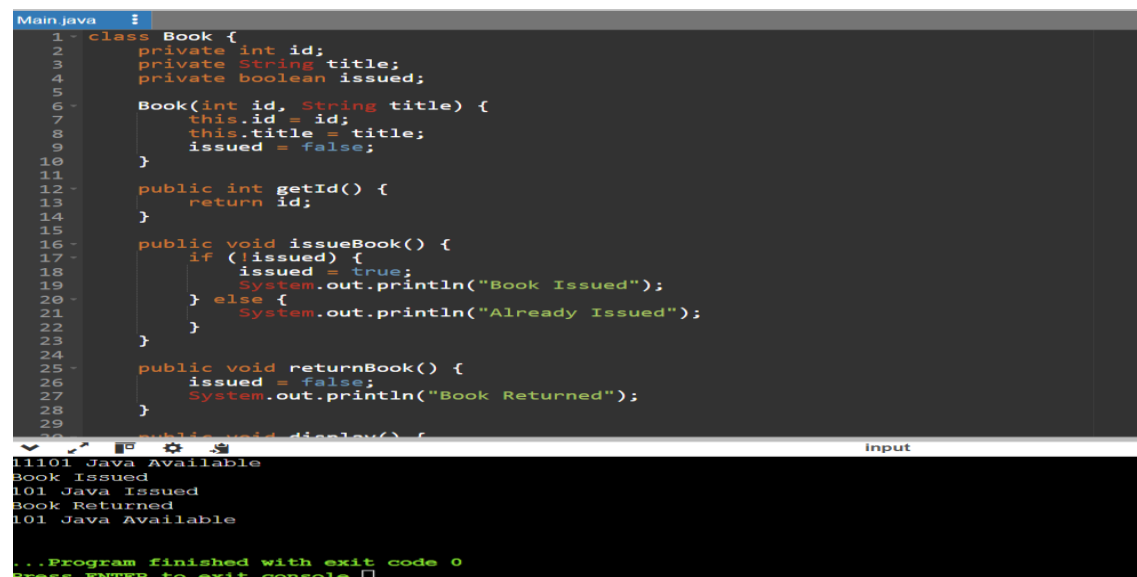
        books[0].display();

    }

}

```

Output:



The screenshot shows a Java IDE with a dark theme. The top pane displays the source code for 'Main.java', which includes the 'Book' class and the 'Main' class. The 'Book' class has attributes 'id', 'title', and 'issued', and methods 'getId()', 'issueBook()', 'returnBook()', and 'display()'. The 'Main' class has a 'main' method that creates an array of 'Book' objects, issues a book, returns it, and displays the status. The bottom pane shows the console output, which matches the expected behavior: '101 Java Available', 'Book Issued', '101 Java Issued', 'Book Returned', and '101 Java Available'. The console also shows the program finished with exit code 0.

```

Main.java
1 class Book {
2     private int id;
3     private String title;
4     private boolean issued;
5
6     Book(int id, String title) {
7         this.id = id;
8         this.title = title;
9         issued = false;
10    }
11
12    public int getId() {
13        return id;
14    }
15
16    public void issueBook() {
17        if (issued) {
18            issued = true;
19            System.out.println("Book Issued");
20        } else {
21            System.out.println("Already Issued");
22        }
23    }
24
25    public void returnBook() {
26        issued = false;
27        System.out.println("Book Returned");
28    }
29
30    public void display() {
31
32
33    }
34 }
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

Output:
101 Java Available
Book Issued
101 Java Issued
Book Returned
101 Java Available
...Program finished with exit code 0
Press ENTER to exit console.

```

2. Employee Payroll System using Inheritance

Design a payroll system with a base class Employee and derived classes PermanentEmployee and ContractEmployee. Override a method calculateSalary() in each subclass to compute salary differently. Display employee details and salary using runtime polymorphism. Also, construct suitable test cases to validate salary calculation for different employee types. BTL: BL3 (Apply)

CODE:

```
class Employee {

    String name;

    Employee(String name) {

        this.name = name;

    }

    double calculateSalary() {

        return 0;

    }

    void display() {

        System.out.println("Name: " + name);

        System.out.println("Salary: " + calculateSalary());

    }

}

class PermanentEmployee extends Employee {

    PermanentEmployee(String name) {

        super(name);

    }

    @Override

    double calculateSalary() {

        return 30000;

    }

}

class ContractEmployee extends Employee {

    ContractEmployee(String name) {
```

```

        super(name);
    }

    @Override

    double calculateSalary() {

        return 15000;

    }

}

public class Main {

    public static void main(String[] args) {

        Employee e1 = new PermanentEmployee("Rahul");

        Employee e2 = new ContractEmployee("Priya")

        e1.display();

        System.out.println();

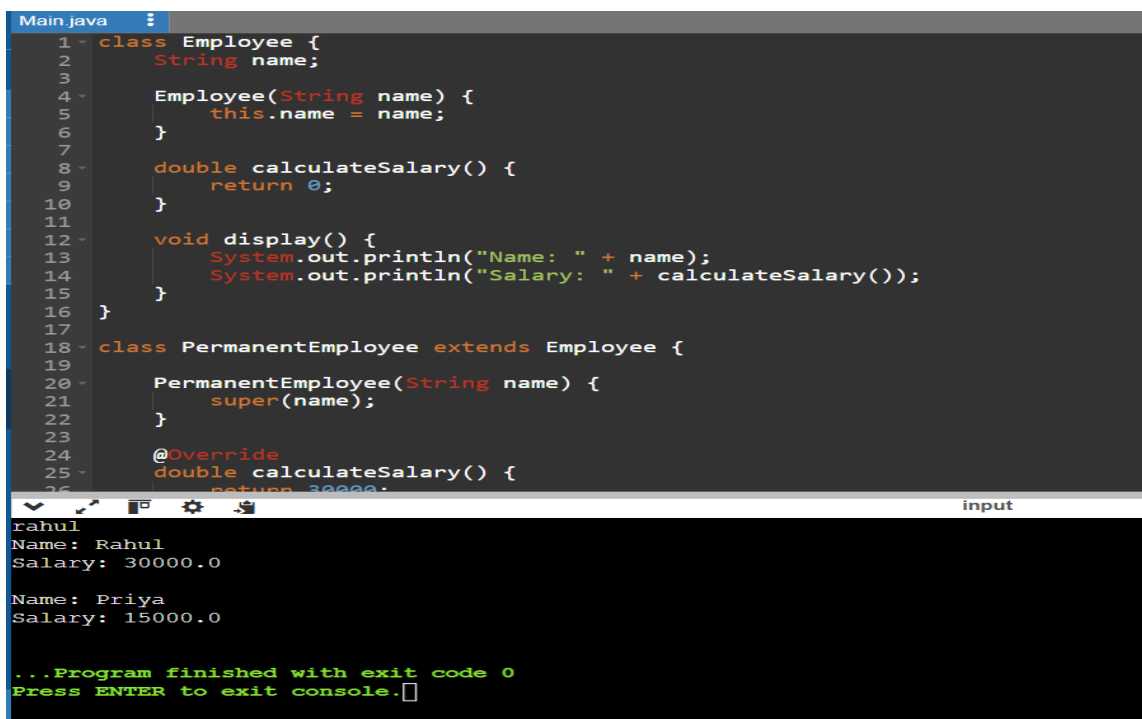
        e2.display();

    }

}

```

Output:



```

Main.java
1 - class Employee {
2 -     String name;
3 -
4 -     Employee(String name) {
5 -         this.name = name;
6 -     }
7 -
8 -     double calculateSalary() {
9 -         return 0;
10 -    }
11 -
12 -    void display() {
13 -        System.out.println("Name: " + name);
14 -        System.out.println("Salary: " + calculateSalary());
15 -    }
16 - }
17 -
18 - class PermanentEmployee extends Employee {
19 -
20 -     PermanentEmployee(String name) {
21 -         super(name);
22 -     }
23 -
24 -     @Override
25 -     double calculateSalary() {
26 -         return 30000;
27 -     }
28 - }

```

```

rahul
Name: Rahul
Salary: 30000.0

Name: Priya
Salary: 15000.0

...Program finished with exit code 0
Press ENTER to exit console.

```

3. Vehicle Rental System using Runtime Polymorphism

Develop a vehicle rental system with a superclass Vehicle and subclasses Car, Bike, and Bus. Override the method calculateRent(int days) in each subclass. Store vehicles in an array of Vehicle references and generate rental bills dynamically. Also, construct suitable test cases to validate rent calculation for different vehicle categories. BTL: BL4 (Analyze)

CODE:

```
class Vehicle {  
  
    double calculateRent(int days) {  
  
        return 0;  
  
    }  
  
}  
  
class Car extends Vehicle {  
  
    @Override  
  
    double calculateRent(int days) {  
  
        return days * 1000;  
  
    }  
  
}  
  
class Bike extends Vehicle {  
  
    @Override  
  
    double calculateRent(int days) {  
  
        return days * 500;  
  
    }  
  
}  
  
class Bus extends Vehicle {  
  
    @Override  
  
    double calculateRent(int days) {  
  
        return days * 2000;  
  
    }  
  
}  
  
public class Main {
```

```

public static void main(String[] args) {

    Vehicle v;

    v = new Car();

    System.out.println("Car Rent for 3 days = " + v.calculateRent(3));

    v = new Bike();

    System.out.println("Bike Rent for 3 days = " + v.calculateRent(3));

    v = new Bus();

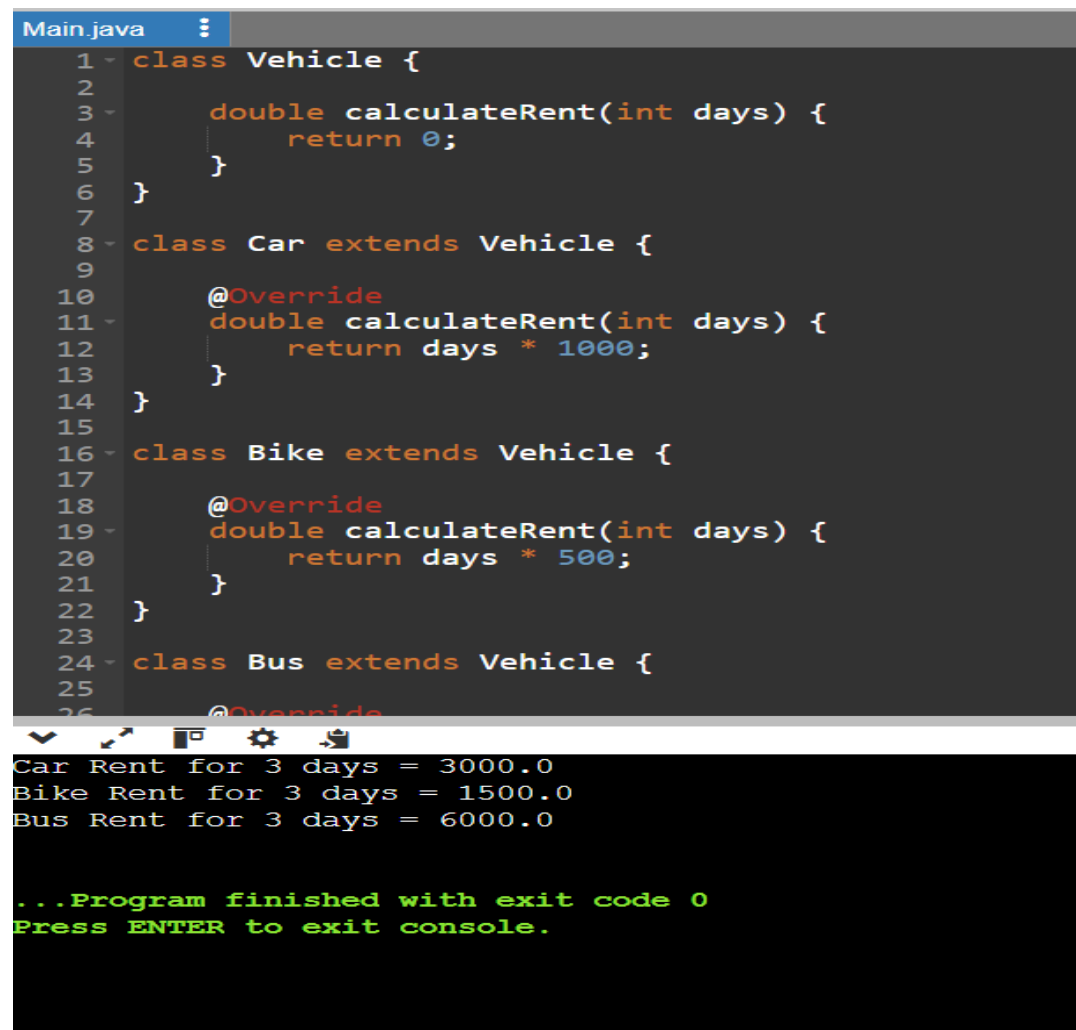
    System.out.println("Bus Rent for 3 days = " + v.calculateRent(3));

}

}

```

Output:



The screenshot shows a Java IDE with a file named 'Main.java'. The code defines a base class 'Vehicle' with a 'calculateRent' method that returns 0. It then defines three subclasses: 'Car', 'Bike', and 'Bus', each overriding 'calculateRent' to return a specific value based on the number of days (3 days * 1000 for Car, 3 days * 500 for Bike, and 3 days * 2000 for Bus). The output window shows the results of running the program: 'Car Rent for 3 days = 3000.0', 'Bike Rent for 3 days = 1500.0', and 'Bus Rent for 3 days = 6000.0'. The program finished with exit code 0.

```

Main.java
1 class Vehicle {
2
3     double calculateRent(int days) {
4         return 0;
5     }
6 }
7
8 class Car extends Vehicle {
9
10    @Override
11    double calculateRent(int days) {
12        return days * 1000;
13    }
14 }
15
16 class Bike extends Vehicle {
17
18    @Override
19    double calculateRent(int days) {
20        return days * 500;
21    }
22 }
23
24 class Bus extends Vehicle {
25
26    @Override

```

```

Car Rent for 3 days = 3000.0
Bike Rent for 3 days = 1500.0
Bus Rent for 3 days = 6000.0

...Program finished with exit code 0
Press ENTER to exit console.

```

4. Bank Account System using Data Hiding

Design a BankAccount class with private fields accountNumber, holderName, and balance. Implement methods for deposit, withdrawal, and balance inquiry with proper validation. Prevent direct modification of balance from outside the class. Also, construct suitable test cases to validate valid and invalid transactions. BTL: BL3 (Apply)

Code;

```
class BankAccount {

    private int accountNumber;

    private String holderName;

    private double balance;

    BankAccount(int accountNumber, String holderName, double balance) {

        this.accountNumber = accountNumber;

        this.holderName = holderName;

        this.balance = balance;

    }

    void deposit(double amount) {

        balance = balance + amount;

        System.out.println("Deposited: " + amount);

    }

    void withdraw(double amount) {

        if (amount <= balance) {

            balance = balance - amount;

            System.out.println("Withdrawn: " + amount);

        } else {

            System.out.println("Insufficient Balance");

        }

    }

    void displayBalance() {

        System.out.println("Account No: " + accountNumber);

        System.out.println("Holder Name: " + holderName);

    }

}
```



```

        System.out.println("Balance: " + balance);
    }
}

public class Main {

    public static void main(String[] args) {

        BankAccount b = new BankAccount(1001, "Rahul", 5000);

        b.displayBalance();

        b.deposit(2000);

        b.withdraw(3000);

        b.displayBalance();

    }

}

```

Output:

The screenshot shows a Java IDE with a file named 'Main.java'. The code defines a 'BankAccount' class with private attributes 'accountNumber', 'holderName', and 'balance'. It includes a constructor and three methods: 'deposit', 'withdraw', and 'displayBalance'. The 'Main' class has a 'main' method that creates a 'BankAccount' object 'b' with account number 1001, holder name 'Rahul', and balance 5000. It then calls 'displayBalance()', 'deposit(2000)', 'withdraw(3000)', and 'displayBalance()' again. The output window shows the execution results: 'Account No: 1001', 'Holder Name: Rahul', 'Balance: 5000.0', 'Deposited: 2000.0', 'Withdrawn: 3000.0', 'Account No: 1001', 'Holder Name: Rahul', 'Balance: 4000.0'. The program finishes with exit code 0.

```

Main.java
1 class BankAccount {
2
3     private int accountNumber;
4     private String holderName;
5     private double balance;
6
7     BankAccount(int accountNumber, String holderName, double balance) {
8         this.accountNumber = accountNumber;
9         this.holderName = holderName;
10        this.balance = balance;
11    }
12
13    void deposit(double amount) {
14        balance = balance + amount;
15        System.out.println("Deposited: " + amount);
16    }
17
18    void withdraw(double amount) {
19        if (amount <= balance) {
20            balance = balance - amount;
21            System.out.println("Withdrawn: " + amount);
22        } else {
23            System.out.println("Insufficient Balance");
24        }
25    }
26
}

Account No: 1001
Holder Name: Rahul
Balance: 5000.0
Deposited: 2000.0
Withdrawn: 3000.0
Account No: 1001
Holder Name: Rahul
Balance: 4000.0

...Program finished with exit code 0
Press ENTER to exit console.

```

5. University Admission System using Method Overloading

Create an Admission class that overloads the method calculateFee() for:

- Undergraduate students,
- Postgraduate students,
- Scholarship students.

Display the fee structure based on user input and compare the overloaded method executions. Also, construct suitable test cases to validate each overloaded version. BTL: BL3 (Apply)

Code:

```
class Admission {

    void calculateFee(int fee) {

        System.out.println("Undergraduate Fee: " + fee);

    }

    void calculateFee(double fee) {

        System.out.println("Postgraduate Fee: " + fee);

    }

    void calculateFee(int fee, int scholarship) {

        int finalFee = fee - scholarship;

        System.out.println("Scholarship Student Fee: " + finalFee);

    }

}

public class Main {

    public static void main(String[] args) {

        Admission a = new Admission();

        a.calculateFee(50000);    // Undergraduate

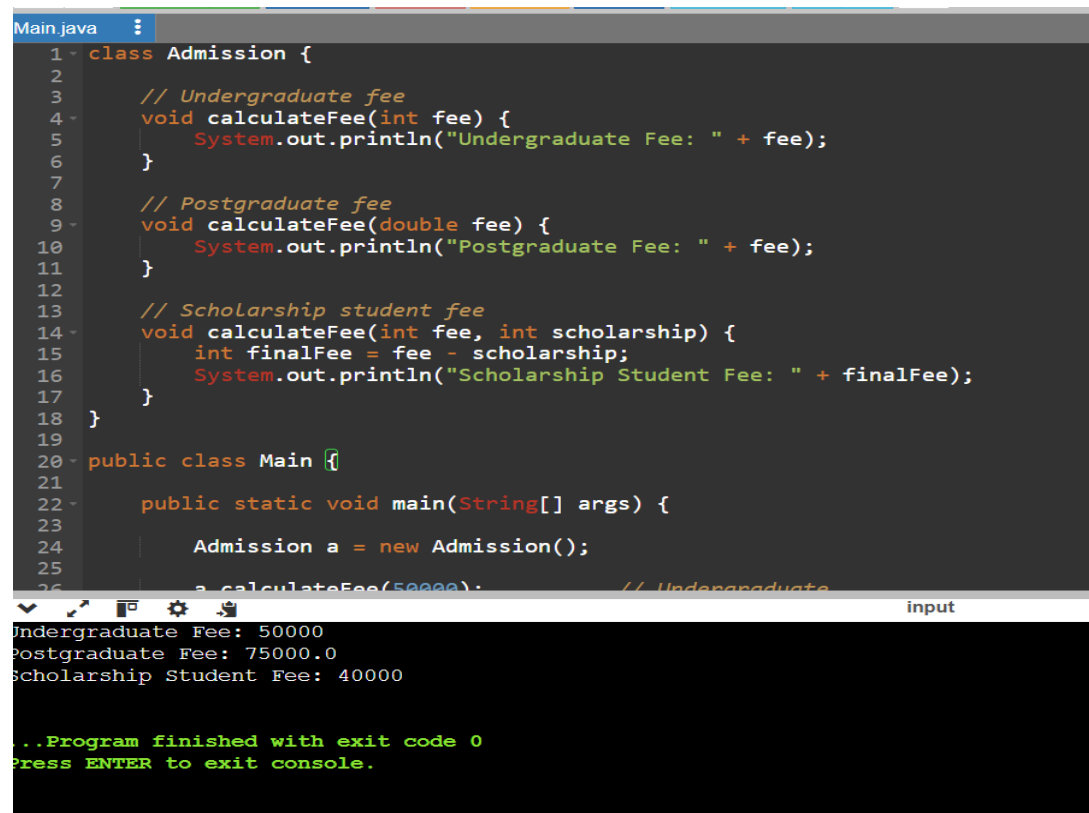
        a.calculateFee(75000.0); // Postgraduate

        a.calculateFee(50000, 10000); // Scholarship Student

    }

}
```

Output:



```
Main.java
1 class Admission {
2
3     // Undergraduate fee
4     void calculateFee(int fee) {
5         System.out.println("Undergraduate Fee: " + fee);
6     }
7
8     // Postgraduate fee
9     void calculateFee(double fee) {
10        System.out.println("Postgraduate Fee: " + fee);
11    }
12
13    // Scholarship student fee
14    void calculateFee(int fee, int scholarship) {
15        int finalFee = fee - scholarship;
16        System.out.println("Scholarship Student Fee: " + finalFee);
17    }
18 }
19
20 public class Main {
21
22     public static void main(String[] args) {
23
24         Admission a = new Admission();
25         a.calculateFee(50000); // Undergraduate
26         a.calculateFee(75000.0); // Postgraduate
27         a.calculateFee(40000, 0); // Scholarship Student
```

Undergraduate Fee: 50000
Postgraduate Fee: 75000.0
Scholarship Student Fee: 40000

...Program finished with exit code 0
Press ENTER to exit console.

6. Shape Area Calculator using Abstract Class & Polymorphism

Design an abstract class Shape with an abstract method area(). Implement subclasses Circle, Rectangle, and Triangle. Use an array of Shape references to compute and display areas dynamically. Also, construct suitable test cases to validate area calculations and polymorphic behavior. BTL: BL4 (Analyze)

Code:

```
abstract class Shape {

    abstract double area();

}

class Circle extends Shape {

    double r;

    Circle(double r) {

        this.r = r;

    }

    @Override
```

```

    double area() {

        return 3.14 * r * r;

    }
}

class Rectangle extends Shape {

    double l, b;

    Rectangle(double l, double b) {

        this.l = l;

        this.b = b;

    }

    @Override

    double area() {

        return l * b;

    }

}

class Triangle extends Shape {

    double b, h;

    Triangle(double b, double h) {

        this.b = b;

        this.h = h;

    }

    @Override

    double area() {

        return 0.5 * b * h;

    }

}

public class Main {

    public static void main(String[] args) {

        Shape[] s = new Shape[3];

```

```

        s[0] = new Circle(5);

        s[1] = new Rectangle(4, 6);

        s[2] = new Triangle(8, 5);

        System.out.println("Circle Area = " + s[0].area());

        System.out.println("Rectangle Area = " + s[1].area());

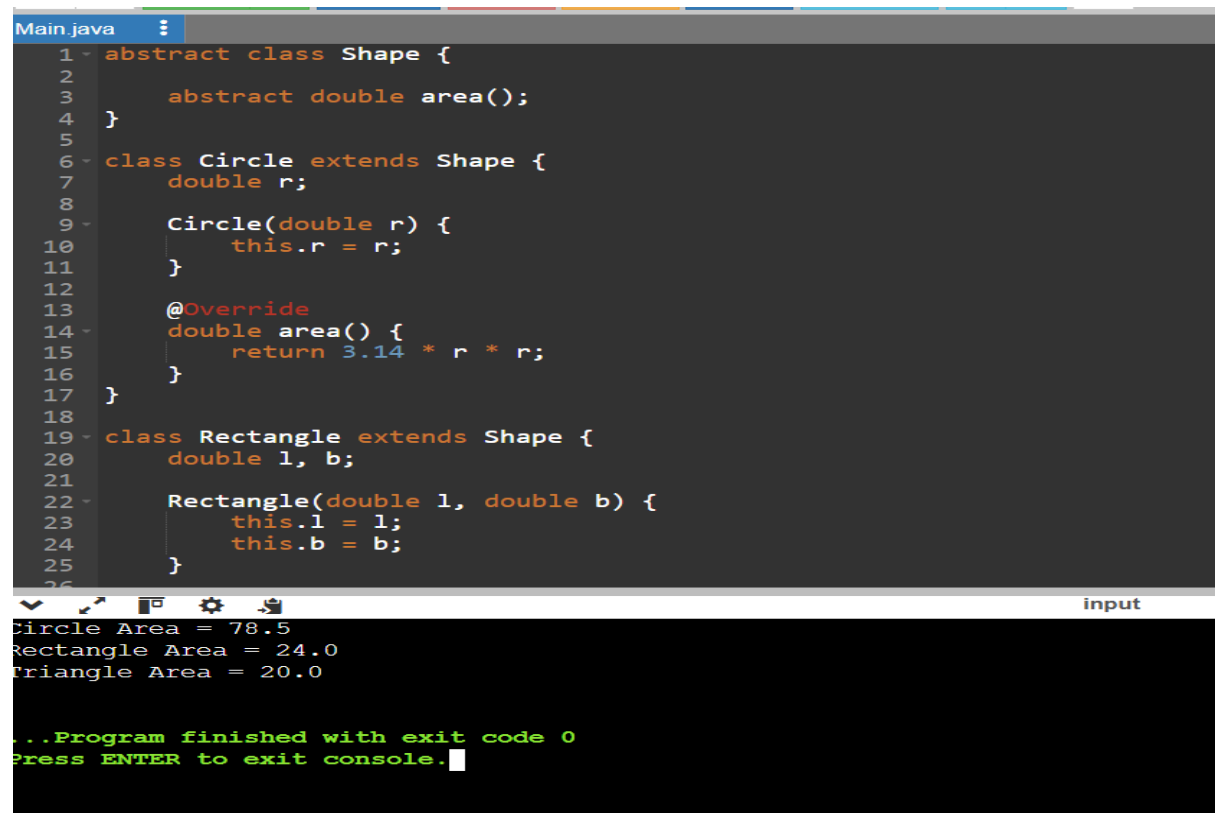
        System.out.println("Triangle Area = " + s[2].area());

    }

}

```

Output:



The screenshot shows a Java IDE with a file named 'Main.java'. The code defines an abstract class 'Shape' with an abstract method 'area()'. Two subclasses, 'Circle' and 'Rectangle', extend 'Shape' and implement the 'area()' method. The 'Circle' class has a radius 'r' and calculates the area as $3.14 * r * r$. The 'Rectangle' class has length 'l' and width 'b' and calculates the area as $l * b$. The main method creates an array 's' of 'Shape' objects, initializes it with a 'Circle' of radius 5, a 'Rectangle' of length 4 and width 6, and a 'Triangle' of base 8 and height 5. It then prints the area of each object.

```

Main.java
1 - abstract class Shape {
2
3     abstract double area();
4 }
5
6 - class Circle extends Shape {
7     double r;
8
9     Circle(double r) {
10         this.r = r;
11     }
12
13     @Override
14     double area() {
15         return 3.14 * r * r;
16     }
17 }
18
19 - class Rectangle extends Shape {
20     double l, b;
21
22     Rectangle(double l, double b) {
23         this.l = l;
24         this.b = b;
25     }
26

```

input

```

Circle Area = 78.5
Rectangle Area = 24.0
Triangle Area = 20.0

...Program finished with exit code 0
Press ENTER to exit console.

```

7. Hospital Management System using Interfaces

Develop a hospital management system with an interface MedicalRecord containing methods addRecord() and displayRecord(). Implement the interface in classes Patient and Doctor. Demonstrate interface-based abstraction and object interaction. Also, construct suitable test cases to validate record creation and display. BTL: BL3 (Apply)

Code:

```
interface MedicalRecord {

    void addRecord();

    void displayRecord();

}

class Patient implements MedicalRecord {

    String name = "Rahul";

    int age = 20;

    public void addRecord() {

        System.out.println("Patient Record Added");

    }

    public void displayRecord() {

        System.out.println("Patient Name: " + name);

        System.out.println("Age: " + age);

    }

}

class Doctor implements MedicalRecord {

    String name = "Dr. Kumar";

    String department = "Cardiology";

    public void addRecord() {

        System.out.println("Doctor Record Added");

    }

    public void displayRecord() {

        System.out.println("Doctor Name: " + name);

        System.out.println("Department: " + department);

    }

}
```

```

    }

}

public class Main {

    public static void main(String[] args) {

        MedicalRecord p = new Patient();

        MedicalRecord d = new Doctor();

        p.addRecord();

        p.displayRecord();

        System.out.println();

        d.addRecord();

        d.displayRecord();

    }

}

```

Output:

```

Main.java
1 interface MedicalRecord {
2     void addRecord();
3     void displayRecord();
4 }
5
6 class Patient implements MedicalRecord {
7     String name = "Rahul";
8     int age = 20;
9
10    public void addRecord() {
11        System.out.println("Patient Record Added");
12    }
13
14    public void displayRecord() {
15        System.out.println("Patient Name: " + name);
16        System.out.println("Age: " + age);
17    }
18 }
19
20 class Doctor implements MedicalRecord {
21     String name = "Dr. Kumar";
22     String department = "Cardiology";
23 }
24
25
26
input
Patient Record Added
Patient Name: Rahul
Age: 20

Doctor Record Added
Doctor Name: Dr. Kumar
Department: Cardiology

...Program finished with exit code 0
Press ENTER to exit console.

```

8. Online Shopping Order System using Packages

Create two packages:

- shopping.product
- shopping.order

Implement classes Product and Order in separate packages. Import the packages into a main application that places orders and calculates the total amount. Also, construct suitable test cases to validate package usage, order placement, and bill generation. BTL: BL3 (Apply)

Code:

```
class Product {  
  
    String name;  
  
    double price;  
  
    Product(String name, double price) {  
  
        this.name = name;  
  
        this.price = price;  
  
    }  
}  
  
class Order {  
  
    void placeOrder(Product p, int quantity) {  
  
        double total = p.price * quantity;  
  
        System.out.println("Product : " + p.name);  
  
        System.out.println("Price  : " + p.price);  
  
        System.out.println("Quantity: " + quantity);  
  
        System.out.println("Total Amount : " + total);  
  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Product p = new Product("Laptop", 50000);  
  
        Order o = new Order();  
  
        o.placeOrder(p, 2);  
  
    }  
}
```

Output:


```
Main.java
1- class Product {
2-     String name;
3-     double price;
4-
5-     Product(String name, double price) {
6-         this.name = name;
7-         this.price = price;
8-     }
9- }
10
11- class Order {
12-
13-     void placeOrder(Product p, int quantity) {
14-         double total = p.price * quantity;
15-
16-         System.out.println("Product : " + p.name);
17-         System.out.println("Price : " + p.price);
18-         System.out.println("Quantity: " + quantity);
19-         System.out.println("Total Amount : " + total);
20-     }
21- }
22
23- public class Main {
24-
25-     public static void main(String[] args) {
26-
27-         Product p = new Product("Laptop", 50000.0);
28-         Order o = new Order();
29-         o.placeOrder(p, 2);
30-     }
31- }
```

Product : Laptop
Price : 50000.0
Quantity: 2
Total Amount : 100000.0

...Program finished with exit code 0
Press ENTER to exit console.

9. Student Result Analyzer using Arrays of Objects

Design a Student class containing roll number, name, and marks in three subjects. Store multiple student objects in an array and implement methods to calculate total, average, grade, and class topper. Also, construct suitable test cases to validate grading and topper identification. BTL: BL3 (Apply)

Code:

```
class Student {

    int rollNo;

    String name;

    int m1, m2, m3;

    Student(int rollNo, String name, int m1, int m2, int m3) {

        this.rollNo = rollNo;

        this.name = name;

        this.m1 = m1;

        this.m2 = m2;

        this.m3 = m3;

    }

    int total() {

        return m1 + m2 + m3;

    }

}
```

```

    }

    double average() {

        return total() / 3.0;

    }

    String grade() {

        double avg = average();

        if (avg >= 90)

            return "A";

        else if (avg >= 75)

            return "B";

        else if (avg >= 50)

            return "C";

        else

            return "Fail";

    }

}

public class Main {

    public static void main(String[] args) {

        Student[] s = {

            new Student(1, "Rahul", 90, 85, 95),

            new Student(2, "Priya", 80, 75, 70),

            new Student(3, "Kiran", 60, 65, 55)

        };

        int max = 0;

        String topper = "";

        for (Student st : s) {

            System.out.println("Roll No : " + st.rollNo);

            System.out.println("Name   : " + st.name);

            System.out.println("Total  : " + st.total());

```

```

        System.out.println("Average : " + st.average());

        System.out.println("Grade  : " + st.grade());

        System.out.println();

        if (st.total() > max) {

            max = st.total();

            topper = st.name;

        }

    }

    System.out.println("Class Topper: " + topper);

    System.out.println("Top Score: " + max);

}

}

```

Output:

```

Main.java
1 class Student {
2
3     int rollNo;
4     String name;
5     int m1, m2, m3;
6
7     Student(int rollNo, String name, int m1, int m2, int m3) {
8         this.rollNo = rollNo;
9         this.name = name;
10        this.m1 = m1;
11        this.m2 = m2;
12        this.m3 = m3;
13    }
14
15    int total() {
16        return m1 + m2 + m3;
17    }
18
19    double average() {
20        return total() / 3.0;
21    }
22
23    String grade() {
24        double avg = average();
25        if (avg >= 90)
26
Roll No : 3
Name : Kiran
Total : 180
Average : 60.0
Grade : C

Class Topper: Rahul
Top Score: 270

...Program finished with exit code 0
Press ENTER to exit console.

```

10. Smart Home Device Controller using Hierarchical Inheritance

Develop a smart home system with a base class Device and derived classes Light, Fan, and AirConditioner. Implement methods to turn devices on/off and display power consumption. Use polymorphism to control devices through a common reference. Also, construct suitable test cases to validate device operations and power calculations. BTL: BL4 (Analyze)

Code:

```
class Device {

    void turnOn() {

        System.out.println("Device is ON");

    }

    void turnOff() {

        System.out.println("Device is OFF");

    }

    double powerConsumption() {

        return 0;

    }

}

class Light extends Device {

    @Override

    double powerConsumption() {

        return 20;

    }

}

class Fan extends Device {

    @Override

    double powerConsumption() {

        return 75;

    }

}

class AirConditioner extends Device {

    @Override

    double powerConsumption() {

        return 1500;

    }

}
```

```
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Device d;  
  
        d = new Light();  
  
        d.turnOn();  
  
        System.out.println("Light Power: " + d.powerConsumption() + " Watts");  
  
        d.turnOff();  
  
        System.out.println();  
  
        d = new Fan();  
  
        d.turnOn();  
  
        System.out.println("Fan Power: " + d.powerConsumption() + " Watts");  
  
        d.turnOff();  
  
        System.out.println();  
  
        d = new AirConditioner();  
  
        d.turnOn();  
  
        System.out.println("AC Power: " + d.powerConsumption() + " Watts");  
  
        d.turnOff();  
  
    }  
  
}
```

Output:

```
Main.java
1 - class Device {
2
3 -     void turnOn() {
4         System.out.println("Device is ON");
5     }
6
7 -     void turnOff() {
8         System.out.println("Device is OFF");
9     }
10
11 -     double powerConsumption() {
12         return 0;
13     }
14 }
15
16 - class Light extends Device {
17
18     @Override
19 -     double powerConsumption() {
20         return 20;
21     }
22 }
23
24 - class Fan extends Device {
25
26     @Override
27
28     double powerConsumption() {
29         return 75;
30     }
31 }
32
33 - class AC extends Device {
34
35     @Override
36     double powerConsumption() {
37         return 1500;
38     }
39 }
40
41 - public class Main {
42     public static void main(String[] args) {
43         Device device = new Device();
44         device.turnOn();
45         System.out.println("Fan Power: 75.0 Watts");
46         device.turnOff();
47
48         Device device2 = new Light();
49         device2.turnOn();
50         System.out.println("AC Power: 1500.0 Watts");
51         device2.turnOff();
52     }
53 }
54
55 ...Program finished with exit code 0
56 Press ENTER to exit console.
```